If you'd rather be notified when a child exits (instead of just waiting around), you can set up a signal handler using trap. Here we set up a trap on SIGCLD, which is the signal sent on "death of child process":

```
trap("CLD") do
  pid = Process.wait
  puts "Child pid #{pid}: terminated"
end

spawn("sort testfile > output.txt")

 # Do other stuff...
```

produces:

```
Child pid 41238: terminated
```

Blocks and Subprocesses

The IO.popen method takes a command as an argument and an optional block. It runs the command and returns an IO object attached to that command. The method then passes the IO object to the block, where you can read from it or (more rarely) write to it.

```
IO.popen("date") { |f| puts "Date is #{f.gets}" }
```

produces:

```
Date is Sun Dec 25 12:30:21 CST 2022
```

The IO object will be closed automatically when the code block exits, just as it is with IO.open.

If you associate a block with Object#fork, the code in the block will be run in a Ruby subprocess, and the parent will continue after the block:

```
fork do
  puts "In child, pid = #$$"
  exit 99
end
pid = Process.wait
puts "Child terminated, pid = #{pid}, status = #{$?.exitstatus}"
```

produces:

```
In child, pid = 41269
Child terminated, pid = 41269, status = 99
```

The $$ here is a global variable that is the process id of the running process.

The wait method will, by default, wait for any subprocess to complete, but you can pass it a process id (pid) as an argument if you want to wait on a specific process.

$? is a global variable that contains information on the termination of a subprocess.

Although Ruby's thread utilities are powerful, they are kind of low-level, and there are some common usage patterns. Ruby gives us two built-in higher-level patterns to support common usage: fibers and Ractors. We'll talk about fibers first.

Creating Fibers

Although the name "fibers" suggests some kind of lightweight thread, Ruby's fibers are just a mechanism for denoting a block of code that can be stopped and restarted, which is